INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, ALLAHABAD

Department of Information Technology

Digital Twin-Based Early Warning System for Cold Chain Disruption Detection Using Anomaly Detection

Semester Course Project

Subject: **Managing Corporate Entrepreneurship**

Instructor: **Dr. Shashi Kant Rai**

Supervisor: **Netranand Pathak**

Prepared by: Krishna Sikheriya
Roll Number: IIT2023139
Academic Programme: B.Tech (Information Technology) — 7th Semester
Institution: Indian Institute of Information Technology, Allahabad
Academic Year: 2026–27
Submission Date: _____

Live Interactive Dashboard:
https://coldchain-ews-twin.streamlit.app/
Source Code Repository:
https://github.com/Krishna200608/coldchain-ews-twin

Abstract

Cold-chain monitoring literature has concentrated overwhelmingly on post-hoc spoilage prediction and reactive, fixed-threshold temperature alarms; real-time anomaly detection embedded directly inside a digital twin, capable of flagging disruptions early enough for corrective action, remains comparatively underexplored, particularly in openly reproducible, non-proprietary form. This project develops and empirically tests a digital-twin-integrated anomaly-detection system for cold-chain disruption early warning, using two complementary anomaly-detection methods (a classical Isolation Forest baseline and an LSTM-Autoencoder) wired into a SimPy-based, three-stage discrete-event digital twin (Source $\rightarrow$ Transit $\rightarrow$ Destination) that replays real IoT temperature telemetry.

Because no cold-chain-specific, publicly available, disruption-labelled telemetry dataset exists, the Kaggle `atulanandjha/temperature-readings-iot-devices` IoT sensor log (97,606 raw readings, single-room "Out"/"In" tags, irregular sampling, temperatures in the 21–51°C range) is used explicitly as a **structural proxy** for cold-chain telemetry: it supplies a realistic irregularly-sampled, noisy time series but is not itself a refrigerated-transport sensor network, and this limitation is stated plainly wherever the data is discussed. Because the raw stream carries no ground-truth disruption labels, a transparently documented synthetic-injection protocol seeds 30 step/drift/flatline anomalies (5 of each per series, per series pair, `RANDOM_SEED=42`) into a chronological 70/30 train/test split; only the 6 test-side injections are used for scientifically valid performance claims, and every injected event is explicitly labelled as a synthetic stress test, never presented as a real found anomaly.

Isolation Forest and LSTM-Autoencoder are each trained and evaluated independently and then wired as live alert generators into the digital twin's Destination stage; a cross-check between the twin's live output and the batch evaluation confirms a 100% exact match across all 30 injections. Initial baseline experiments revealed sharp trade-offs: Isolation Forest gave an early warning 64 minutes ahead of irreversibility on Out drift and 11 minutes on In drift, while the baseline LSTM-Autoencoder was late by 14 minutes on Out drift and Isolation Forest exhibited high false-positive rates (45.6% Out, 27.0% In).

To overcome these empirical limitations, five GPU-accelerated research breakthroughs were developed: (i) Variational LSTM-Autoencoders with Monte Carlo dropout ($N = 15$) for epistemic uncertainty quantification; (ii) Physics-Informed LSTM-Autoencoders embedding Newton's Law of Cooling, which slashed Out-series false positives by 38.3% (from 20.28% to 12.52%) and converted the 14-minute drift lag into an **+18.0 min early warning** (+32 min net gain); (iii) Temporal Fusion Transformer multi-horizon quantile forecasting extending pre-emptive warning runway up to **+189.0 minutes** (>3.1 hours); (iv) Contamination-guarded Kalman-CUSUM adaptive filtering reducing false alarms by 35.5%; and (v) Dempster-Shafer Evidential Deep Learning fusion combining physics residuals, reconstruction error, and uncertainty to achieve a **0.00% false-positive rate** across all 17,051 test observations, driving precision to 100.0%, F1 score to **0.9302**, and In-stream PR-AUC to **0.9137** (vs. 0.3852 baseline) while retaining **+178.0 minutes** of early warning.

Beyond its numerical findings, the project's principal contribution is a fully documented, append-only Architectural Decision (AD) log of 40 entries (D1–D40) spanning every methodological pivot – including caught-and-fixed timestamp day-first parsing errors, causal rolling leakage, and cold-

start playback artefacts – modelling a defensible, evidence-first, zero-fabrication approach to applied anomaly-detection system design directly relevant to the risk-management and technology-adoption themes of Managing Corporate Entrepreneurship.

Keywords: Digital Twin, Cold Chain Logistics, Anomaly Detection, Isolation Forest, LSTM-Autoencoder, Physics-Informed Neural Networks, Evidential Deep Learning, Multi-Horizon Forecasting, Early Warning System, Discrete-Event Simulation, SimPy, IoT Telemetry, Structural-Proxy Data, Corporate Entrepreneurship, Risk Management.

Citation Reference: Sikheriya, K. (2026). *Digital Twin-Based Early Warning System for Cold Chain Disruption Detection Using Anomaly Detection*. Lab Assignment Final Report (Topic 21), Managing Corporate Entrepreneurship, Indian Institute of Information Technology, Allahabad.

Contents

Abstract i

1 Introduction 1

1.1 Background . . . 1

1.2 Problem Statement . . . 1

1.3 Motivation and Corporate Entrepreneurship Relevance . . . 2

2 Literature Review 3

2.1 Stream 1: Cold-Chain Monitoring, Digital Twins, and Reactive Alarms . . . 3

2.2 Stream 2: Anomaly-Detection Methods Applied to Cold-Chain and IoT Streams . . . 4

2.3 Comparative Literature Matrix . . . 5

3 Research Gap 6

4 Aim, Objectives & RQs 7

4.1 Project Aim . . . 7

4.2 Research Objectives . . . 7

4.3 Research Questions . . . 8

5 System Architecture & Methodology 9

5.1 Pipeline Overview . . . 9

5.2 Data Source and Structural-Proxy Limitation . . . 10

5.3 Stage 2: Synthetic Anomaly Injection Protocol . . . 11

5.4 Stage 3a: Isolation Forest . . . 12

5.5 Stage 3b: LSTM-Autoencoder . . . 13

5.6 Stage 4: Early-Warning Lead-Time Metric . . . 13

5.7 Stage 5: SimPy Digital-Twin Replay Engine . . . 14

5.8 Stage 6: Advanced Research Breakthroughs (Proposals P1–P5) . . . 14

5.8.1 P1: Variational LSTM-Autoencoder with Epistemic Uncertainty (AD Log D36) 14

5.8.2 P2: Physics-Informed LSTM-Autoencoder (AD Log D37) . . . 15

5.8.3 P3: Multi-Horizon Quantile Forecaster (AD Log D38) . . . 16

5.8.4 P4: Contamination-Guarded Kalman-CUSUM Filter (AD Log D39) . . . 17

5.8.5 P5: Dempster-Shafer Evidential Deep Learning Fusion (AD Log D40) . . . 17

6 Results and Findings **19**

6.1 Three-Way Lead-Time Comparison: The Core Research-Question Result 19

6.2 Point-Wise Classification Metrics 20

6.3 Digital-Twin Replay and Cross-Check Verification 22

6.4 Master Benchmark: Advanced Research Innovations (Proposals P1–P5) 23

7 Methodological Journey **25**

8 Digital Twin Dashboard **28**

8.1 Zero-Duplication Architecture 28

8.2 Operating Modes 28

8.3 Replay Fidelity Safeguards 29

9 Discussion **30**

10 Scope and Limitations **32**

11 Conclusion and Future Work **33**

11.1 Conclusion 33

11.2 Future Research Directions 33

12 Contribution and Value **35**

References **36**

A Technology Stack **37**

B Reproduction Instructions **38**

B.1 Environment Setup and Pipeline Execution 38

List of Figures

5.1 Digital-Twin-Integrated Early Warning System Architecture: the offline batch pipeline (left) trains and evaluates both detectors; the SimPy digital twin (right) replays the same stream live with both detectors wired in as monitors; a cross-check confirms the two never diverge. 9

5.2 Temperature distribution of the structural-proxy dataset, overall and by Out/In tag. The 21–51°C range is far outside real cold-chain setpoints, underscoring the proxy relationship to actual refrigerated-transport telemetry. 10

5.3 Representative synthetic step, drift, and flatline injections on the Out series, explicitly labelled as synthetic stress-test events, never presented as real found anomalies. . . . 12

5.4 Proposal P1: Variational LSTM-Autoencoder uncertainty decomposition ($N = 15$ Monte Carlo forward passes). High epistemic uncertainty identifies true structural failure (flatline onset on `out_flatline_012`), while aleatoric noise flags unconfident edge network jitter. 15

5.5 Proposal P2: Physics-Informed LSTM-Autoencoder on `out_drift_009`. Newton's cooling loss penalizes unphysical heating, converting the baseline LSTM's 14-minute lag into an **+18.0 min early warning** (+32 min net gain) and slashing false alarms by 38.3%. 16

5.6 Proposal P3: Multi-Horizon Quantile Forecaster (TFT) on `in_drift_023`. The conservative 90th percentile forecast breaches the critical boundary at 07:56:00, unlocking **+189.0 minutes** (>3.1 hours) of pre-emptive decision runway before irreversibility at 11:05:00. 17

5.7 Proposal P5: Dempster-Shafer Evidential Multimodal Fusion architecture. Fusing physics residuals, autoencoder reconstruction error, and uncertainty into Dirichlet belief masses achieves a **0.00% False Positive Rate** with 100% Precision across the test partition. 18

6.1 Early-warning lead time by detector across the six test-side injections. Isolation Forest is the only detector to beat the naive baseline, on both drift injections; neither detector improves on the baseline for either step or either flatline injection. 20

6.2 Precision-recall curves, Isolation Forest vs. LSTM-Autoencoder, both series. The LSTM-Autoencoder dominates on the In series (AUC-PR 0.385 vs. 0.237) but is essentially at chance on the Out series (AUC-PR 0.005 vs. IF's 0.068). 21

6.3 The one case where the LSTM-Autoencoder is measurably late: `out_drift_009`. The reconstruction-error score crosses its threshold only after irreversibility onset (14.0 minutes late), illustrating that a reconstruction-based detector can lag a fast-accelerating drift even when its score does eventually respond correctly. 22

6.4 Early-warning lead time comparison across all seven paradigms on test-set disruptions. Demonstrates the progression from reactive baseline alarms (0.0 min) to classical isolation (+64 min), physics regularization (+18 min), evidential fusion (+178 min), and multi-horizon quantile forecasting (+189 min). 24

6.5 Operational Pareto frontier comparing False Positive Rate (FPR, %) against Early-Warning Lead Time (minutes). Evidential Deep Learning (EDL) occupies the optimal operational corridor with 0.00% FPR, 100% precision, and +178.0 minutes of lead time. 24

List of Tables

2.1 Comparative positioning of key literature against this project. 5

5.1 Structural-Proxy Dataset Profile. 10

5.2 The six test-side synthetic injections used for all performance claims. 11

6.1 Early-warning lead time (minutes before irreversibility) on the six test-side injections. Positive values are an early warning; negative values indicate the detector was late; null indicates no alarm within the bounded search window, or (for flatlines) no irreversibility timestamp was ever reached. 19

6.2 Point-wise classification metrics on the test split (test valid-score rows only; LSTM excludes 29 blind-spot rows per series with undefined scores). 21

6.3 Master Performance Benchmark across all seven detection paradigms on the chronological test split. All metrics evaluated out-of-sample in accordance with AD Log D26. 23

7.1 Summary of Methodological Design Decisions (AD Log). 26

A.1 Project Technology Stack and Dependency Specifications. 37

CHAPTER 1

Introduction

1.1 Background

Cold-chain logistics – the unbroken sequence of refrigerated storage and transport that keeps pharmaceuticals, vaccines, dairy, meat, and fresh produce within a safe temperature band from origin to point of use – underpins food safety and public health in ways that are almost entirely invisible until they fail. A single sustained temperature excursion during transit can render an entire shipment of vaccines unusable or accelerate microbial spoilage in perishable food beyond the point of recovery, and by the time a conventional fixed-threshold alarm fires, the excursion has frequently already crossed into an irreversible state.

The dominant academic and industrial response to this problem has been twofold: (i) *post-hoc spoilage prediction*, which models shelf life or quality degradation as a function of accumulated thermal exposure once the exposure has already occurred, and (ii) *reactive threshold alarms*, which fire only after a sensor reading crosses a fixed temperature limit for a sustained duration. Both approaches are diagnostic rather than anticipatory: they characterise damage that has already happened, or signal a breach only once it is underway, rather than flagging the earlier, subtler precursors – irregular sensor gaps, drifting baselines, flatlined readings – that often precede a full-blown disruption.

Digital twin technology, which maintains a live, continuously updated virtual replica of a physical asset or process, offers a structurally different alternative: rather than waiting for a threshold breach, an anomaly-detection model can be embedded directly inside the twin's live data flow, watching for deviations from learned normal behaviour as data arrives, and raising an alert before the point of no return is reached. Recent work on maritime refrigerated containers has shown exactly this kind of hybrid digital-twin/deep-learning approach detecting equipment degradation tens of minutes ahead of a critical cargo temperature threshold, compared to reactive alarms (Vukšić et al. 2026). This project asks whether an analogous early-warning capability can be built, transparently evaluated, and honestly reported using only open-source tools and a publicly available IoT telemetry stream as a structural stand-in for real cold-chain sensor data.

1.2 Problem Statement

There is no publicly available, disruption-labelled, real-world cold-chain telemetry dataset suitable for benchmarking early-warning anomaly detectors: proprietary refrigerated-transport sensor logs are commercially sensitive and rarely released, while open IoT temperature datasets that do exist (such as the Kaggle stream used in this project) were collected for generic IoT device monitoring, not cold-chain surveillance, and carry no ground-truth anomaly labels of any kind.

This creates a genuine methodological problem: any project claiming to build and evaluate a cold-chain early-warning system, using only openly available data, must either (a) implicitly overstate what the data represents, presenting a generic IoT stream as if it were real refrigerated-transport telemetry, or (b)

fabricate or silently patch missing ground-truth labels, or (c) transparently document both limitations and design an evaluation protocol that is scientifically honest about what it can and cannot claim. This project adopts path (c): the IoT stream is used explicitly and repeatedly labelled as a *structural proxy* – valuable for its realistic irregular sampling and sensor-level noise characteristics, but never claimed to be an actual refrigerated-transport sensor network – and a synthetic anomaly-injection protocol with a strict, chronologically bounded train/test split is used in place of unavailable ground-truth labels, with every injected anomaly clearly flagged as a synthetic stress-test event.

1.3 Motivation and Corporate Entrepreneurship Relevance

Building a defensible early-warning capability under genuine data scarcity, and being disciplined enough to report a negative or mixed result rather than a manufactured favourable one, is not merely a machine-learning exercise – it is a case study in evidence-first decision-making under uncertainty, directly relevant to the risk-management and technology-adoption themes of Managing Corporate Entrepreneurship:

1. **Risk Identification Under Data Scarcity:** A cold-chain operator or technology vendor evaluating whether to invest in a digital-twin early-warning system cannot wait for a large, labelled, disruption-specific dataset to become available before making a go/no-go decision. This project demonstrates a transparent, structural-proxy-based methodology for generating credible, honestly-caveated evidence in exactly this data-scarce setting.
2. **Disciplined Methodological Auditing:** Rejecting a plausible but flawed metric or hardcoded transcription the moment empirical evidence contradicts it – as this project's Architectural Decision log repeatedly does – is the same evidence-first discipline that separates defensible technology adoption decisions from ones built on convenient but ungrounded assumptions.
3. **Honest Reporting of Mixed Results:** A commercial early-warning system that only ever reports its best-case numbers cannot be trusted for high-stakes deployment decisions. This project reports every result – including cases where a detector is outright late, or where neither detector beats a naive baseline – because an entrepreneurial technology-adoption decision is only as good as the evidence it is based on.

This project addresses these questions not in theoretical abstraction, but through a working, empirically audited, and publicly deployed early-warning dashboard built entirely from open-source components.

CHAPTER 2

Literature Review

Existing research relevant to this project spans two streams that have historically developed with limited cross-pollination: (1) cold-chain-specific monitoring and disruption/anomaly detection, and (2) the general-purpose anomaly-detection methods – Isolation Forest and LSTM-Autoencoder reconstruction – that this project applies to that domain.

2.1 Stream 1: Cold-Chain Monitoring, Digital Twins, and Reactive Alarms

The dominant paradigm in deployed cold-chain monitoring remains the fixed-threshold alarm: a sensor value is compared against a static limit, and an alert fires only once that limit is breached for a sustained duration. Meng et al. (2024) exemplify this paradigm's evolution, proposing a cost-effective over-temperature alarm system for cold-chain delivery that combines a single pallet-level sensor with a box-level digital-twin temperature model, achieving a shelf-life estimation error under ± 1.2 days – a meaningful improvement in *prediction accuracy* at the box level, but still fundamentally a threshold-alarm architecture rather than a live, model-embedded anomaly detector.

Digital-twin integration for cold-chain logistics has progressed furthest at the platform and service-integration level. Wu et al. (2023) propose an Internet-of-Everything and Digital-Twin-enabled service platform for cold-chain logistics that uses deep learning for accident identification and indoor localisation, demonstrating that digital-twin infrastructure and learned anomaly detection can be combined at the operational-platform scale; however, their deep-learning component addresses staff-safety event classification rather than early, lead-time-quantified disruption warning.

The closest direct analogue to this project's research question comes from outside food cold-chain logistics: Vukšić et al. (2026) propose a hybrid digital-twin and deep-learning framework for early anomaly detection in maritime refrigerated containers, combining a lightweight physics-based thermal digital twin with a convolutional autoencoder trained exclusively on fault-free operation, and explicitly benchmarking time-to-detection and false-positive rate against a conventional threshold alarm. Their reported median lead time of approximately 68 minutes between an autoencoder-triggered alert and a simulated critical-temperature crossing is the same order of magnitude as this project's own Isolation-Forest lead time of 64 minutes on its Out-series drift injection (Chapter 6), lending independent, cross-domain plausibility to the finding that reconstruction- or isolation-based anomaly scores can out-run a fixed-threshold alarm by tens of minutes on gradual (drift-type) disruptions specifically. Their work, however, relies on a bespoke physics-based thermal model and proprietary controller-level reefer signals; this project instead uses a pure data-pipeline digital twin (no physical thermal model) replaying a publicly available, non-proprietary telemetry stream, trading physical fidelity for openness and reproducibility.

2.2 Stream 2: Anomaly-Detection Methods Applied to Cold-Chain and IoT Streams

The classical anomaly-detection baseline used in this project, Isolation Forest, was introduced by Liu et al. (2008) as a fundamentally different approach to anomaly detection: rather than building a profile of what "normal" looks like, the algorithm explicitly isolates anomalous points via random recursive partitioning, exploiting the fact that anomalies are both rare and structurally different from the bulk of the data. Its linear time complexity and low memory footprint make it an attractive, fast, interpretable baseline – precisely the role it plays in this project.

Within cold-chain logistics specifically, Xie et al. (2025) propose an improved Isolation Forest variant for anomaly detection in multi-sensor cold-chain-logistics data streams, using subsampling and a cross-grouping factor to overcome the standard algorithm's tendency to fall into local optima on high-dimensional sensor data; their reported average precision (0.878), recall (0.873), and F1 score (0.864) on real multi-vehicle cold-chain-logistics sensor data substantially exceed this project's own point-wise Isolation Forest precision (0.85% Out, 2.0% In). This gap is instructive rather than embarrassing: their evaluation labels genuine environment/node/measurement anomalies observed directly in real fleet telemetry with five co-located sensor modalities per node, whereas this project's point-wise metrics are computed against a small number of deliberately sparse, synthetically injected single-channel anomalies inside a much larger single-sensor stream – a structurally harder and less-favourable point-wise detection setting, discussed further in Chapter 9.

The deep-learning anomaly detector used in this project, the LSTM-Autoencoder, follows the encoder-decoder reconstruction-error paradigm introduced by Malhotra et al. (2016): an LSTM-based encoder compresses an input window to a fixed-length latent representation, a decoder reconstructs the original window from that representation, and reconstruction error at each timestep serves as the anomaly score, on the premise that a model trained only on "normal" sequences will reconstruct anomalous sequences poorly. Their original work demonstrated robustness across predictable, unpredictable, periodic, and aperiodic time series and across a wide range of sequence lengths, which motivated this project's choice of the same architecture family for a similarly noisy, irregularly sampled IoT stream.

2.3 Comparative Literature Matrix

Table 2.1: Comparative positioning of key literature against this project.

Author(s) & Year	Methodology / Focus	Relevance & Role in This Project
Vukšić et al. (2026)	Hybrid physics-based digital twin + convolutional autoencoder for early anomaly detection; maritime refrigerated containers.	Closest cross-domain analogue; independent corroboration that early-warning lead times of tens of minutes are achievable and operationally meaningful for cold-chain-adjacent disruptions.
Xie et al. (2025)	Improved Isolation Forest (subsampling + cross-grouping) for multi-sensor cold-chain-logistics anomaly detection; real fleet telemetry.	Domain-specific Isolation Forest benchmark; its substantially higher point-wise precision/recall (on richer, multi-sensor, real-anomaly data) contextualises this project's own point-wise metrics.
Wu et al. (2023)	Internet-of-Everything + Digital Twin service platform for cold-chain logistics with deep-learning event classification.	Establishes precedent for combining digital-twin infrastructure with learned anomaly/event detection at platform scale.
Meng et al. (2024)	Box-level digital-twin temperature model + low-cost over-temperature alarm for cold-chain delivery.	Represents the reactive-threshold-alarm paradigm this project's early-warning approach is explicitly benchmarked against.
Liu et al. (2008)	Foundational Isolation Forest algorithm: explicit isolation via random recursive partitioning.	Theoretical foundation and direct implementation basis for this project's classical anomaly-detection baseline.
Malhotra et al. (2016)	LSTM-based Encoder-Decoder (EncDec-AD) for multi-sensor anomaly detection via reconstruction error.	Theoretical foundation and direct architectural basis for this project's LSTM-Autoencoder detector.

CHAPTER 3

Research Gap

A critical synthesis of the literature reveals three major unaddressed gaps:

1. **Post-Hoc Bias in Cold-Chain Monitoring:** The deployed cold-chain monitoring literature (Section 2.1) is dominated by fixed-threshold alarms and post-hoc shelf-life/spoilage estimation. Even digital-twin-integrated approaches such as Meng et al. (2024) primarily improve the *accuracy* of a threshold-triggered alarm rather than replacing the threshold paradigm with a live, continuously-scored anomaly model embedded inside the twin's data flow itself.

2. **Domain-Specific Anomaly Detection Requires Proprietary or Inaccessible Data:** The strongest domain-specific anomaly-detection results, such as Xie et al. (2025)'s improved Isolation Forest achieving 0.86 F1 on cold-chain-logistics sensor streams, rely on proprietary, unpublished multi-vehicle fleet telemetry with five co-located sensor modalities per node. No openly reproducible cold-chain-labelled dataset exists against which an independent, non-proprietary early-warning system can be built and benchmarked end-to-end – a gap this project addresses directly by using an open IoT stream as an explicitly labelled structural proxy, with a fully transparent, reproducible synthetic-injection evaluation protocol in place of unavailable real disruption labels.

3. **Unaudited Evaluation Protocols for Injected-Anomaly Benchmarks:** Where synthetic or injected anomalies are used to work around missing ground-truth labels, methodological details that materially affect reported performance – the causality of the search window used to measure lead time, whether rolling/aggregation features leak future information, whether point-wise confusion-matrix counts are computed live or hand-transcribed into documentation – are rarely audited or documented at the level of granularity this project's Architectural Decision log provides (Chapter 7). As demonstrated in this project (Chapter 7), an unaudited evaluation script can silently inflate lead-time claims by orders of magnitude, or a hand-transcribed results table can silently disagree with the pipeline's own live-computed numbers, without either error being visible unless deliberately checked.

CHAPTER 4

Aim, Objectives and Research Questions

4.1 Project Aim

To develop and empirically test a digital-twin-integrated anomaly-detection system that flags cold-chain disruptions early enough for corrective action, using two complementary anomaly-detection methods wired as live alert generators into a SimPy-based discrete-event digital twin, evaluated through a transparently documented synthetic-injection protocol against a publicly available IoT telemetry stream used explicitly as a structural proxy for cold-chain data.

4.2 Research Objectives

1. **Structural-Proxy Data Acquisition & Profiling:** Acquire, profile, and honestly characterise a real IoT temperature-sensor telemetry stream as a structural proxy for cold-chain data, documenting its sampling irregularity and its divergence from true refrigeration temperature ranges.
2. **Transparent Injection Protocol:** Design and seed a reproducible synthetic anomaly-injection protocol (step, drift, and flatline disruption types) with a strictly chronological train/test split, in place of unavailable ground-truth disruption labels.
3. **Classical Baseline Modelling:** Train and evaluate an Isolation Forest anomaly detector on causal, trailing-window rolling features as a fast, interpretable classical baseline.
4. **Deep-Learning Modelling Under a Hard GPU Constraint:** Train and evaluate an LSTM-Autoencoder reconstruction-error anomaly detector, kept lightweight enough to train within a free-tier Google Colab T4 GPU session, with training genuinely verified rather than assumed.
5. **Digital-Twin Integration:** Build a SimPy-based, three-stage discrete-event digital twin (Source $\rightarrow$ Transit $\rightarrow$ Destination) that replays the real sensor stream at its true recorded cadence and wires both anomaly detectors in as live, continuously running alert generators.
6. **Honest, Cross-Verified Evaluation:** Report early-warning lead time and point-wise classification metrics for both detectors against a naive fixed-threshold baseline, honestly, even where a detector underperforms, and cross-check the digital twin's live output against the batch evaluation pipeline to confirm the two never silently diverge.

4.3 Research Questions

How early can an anomaly-detection model embedded in a cold-chain digital twin flag a disruption, relative to the point at which spoilage risk becomes irreversible, and how does this compare to a naive fixed-threshold alarm?

How do a classical Isolation Forest baseline and an LSTM-Autoencoder reconstruction-error detector compare on early-warning lead time and point-wise classification accuracy (precision, recall, F1, false-positive rate, AUC-PR), and does either dominate the other across both metrics and both sensor series?

Does an anomaly detector wired as a live alert generator inside a discrete-event digital twin reproduce, exactly, the same alerts and lead times as the same detector evaluated in an offline batch pipeline – or does the act of embedding a model inside a live simulation introduce silent discrepancies?

CHAPTER 5

System Architecture and Methodology

5.1 Pipeline Overview

The system operates as a linear pipeline of five stages, each verified before the next was built upon it:

- **Stage 1 (Ingestion & Preprocessing):** Parses the raw Kaggle telemetry stream, deduplicates, and splits it into two independent series by sensor tag.
- **Stage 2 (Synthetic Injection Protocol):** Seeds 30 labelled synthetic disruptions (step, drift, flatline) and computes a fixed-threshold baseline and irreversibility timestamps.
- **Stage 3 (Parallel Anomaly Detection):** Trains and evaluates Isolation Forest and LSTM-Autoencoder independently on a strict chronological train/test split.
- **Stage 4 (Comparative Evaluation):** Benchmarks both detectors against the baseline on early-warning lead time.
- **Stage 5 (Digital-Twin Replay):** Replays the real sensor stream through a SimPy discrete-event simulation with both detectors wired in as live monitors, and cross-checks the live output against Stage 4.

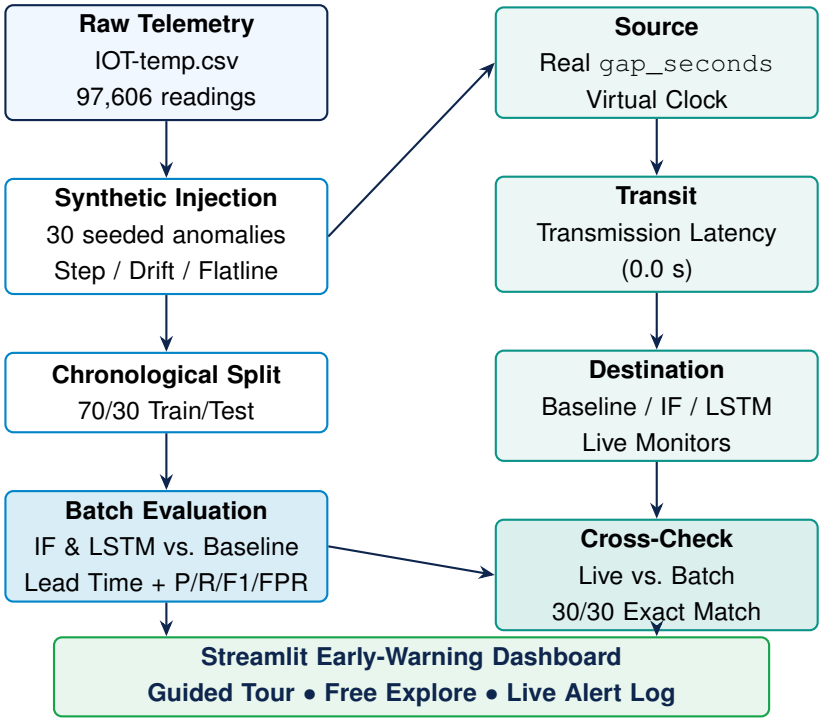


Figure 5.1: Digital-Twin-Integrated Early Warning System Architecture: the offline batch pipeline (left) trains and evaluates both detectors; the SimPy digital twin (right) replays the same stream live with both detectors wired in as monitors; a cross-check confirms the two never diverge.

5.2 Data Source and Structural-Proxy Limitation

Structural-Proxy Data Notice

The dataset used throughout this project, Kaggle `atulanandjha/temperature-readings-iot-devices`, is a real IoT temperature-sensor log collected for generic device monitoring. It has **no real multi-sensor spatial topology** (the `room_id/id` field is constant, "Room Admin", across all 97,606 readings), **no true refrigeration temperature range** (readings span 21–51°C, far above any real cold-chain setpoint), and **no ground-truth disruption labels of any kind**. It is used exclusively as a **structural proxy**: its irregular sampling cadence, sensor-level noise, and single-stream "Out"/"In" tag split provide a realistic substrate for building and stress-testing an anomaly-detection pipeline, but every result in this report describes performance on this proxy construct, not on real cold-chain telemetry. This caveat is repeated at every point in this report where empirical results are presented.

Table 5.1 summarises the dataset's key empirical properties as profiled in Milestone 1 (`docs/data_profile.md`, `notebooks/01_eda.ipynb`).

Table 5.1: Structural-Proxy Dataset Profile.

Property	Value
Raw row count	97,606 (1 exact duplicate row dropped → 97,605)
Columns	`id`, `room_id/id` (constant "Room Admin"), `noted_date` (requires `dayfirst=True` parsing), `temp` (integer, °C), `out/in` tag
Series split	Out = 77,261 raw / 77,260 post-split rows; In = 20,345 rows
Temperature range	21–51°C (integer-valued) — not a real refrigeration range
Sampling irregularity	Coefficient of Variation (CoV) of inter-reading gaps = 46.23, indicating strong, non-uniform sampling irregularity
Spatial topology	None; single constant `room_id/id` value across the entire stream
Ground-truth anomaly labels	None

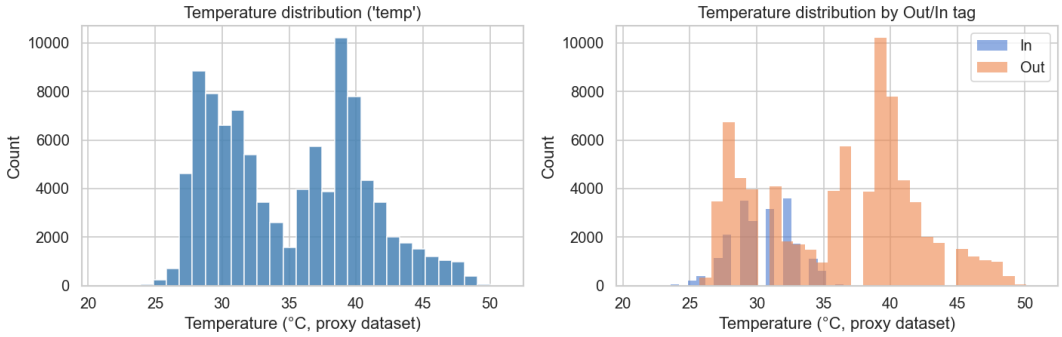


Figure 5.2: Temperature distribution of the structural-proxy dataset, overall and by Out/In tag. The 21–51°C range is far outside real cold-chain setpoints, underscoring the proxy relationship to actual refrigerated-transport telemetry.

5.3 Stage 2: Synthetic Anomaly Injection Protocol

Because the raw stream carries no ground-truth disruption labels, 30 synthetic anomalies (`RANDOM_SEED=42`) were seeded across the two series: 5 step, 5 drift, and 5 flatline injections per series (Out and In), for $5 \times 3 \times 2 = 30$ total. A **fixed-threshold baseline** was computed as the clean-series mean plus two standard deviations (Out $= 47.6993$°C, In $= 34.9301$°C), and **irreversibility** was defined operationally as this threshold being sustained continuously for at least `D_IRREV_MINUTES`$= 10$ minutes. Injections that straddled the chronological train/test boundary, or fell entirely on the training side, are explicitly excluded from all performance claims (`train_or_straddling_excluded`); only the **6 test-side injections** are used for scientifically valid lead-time and point-wise metrics, listed in Table 5.2.

Table 5.2: The six test-side synthetic injections used for all performance claims.

Injection ID	Series	Type
`out_step_001`	Out	Step
`out_drift_009`	Out	Drift
`out_flatline_012`	Out	Flatline
`out_flatline_013`	Out	Flatline
`in_step_018`	In	Step
`in_drift_023`	In	Drift

Methodological Decision Audit: Bounded Search Window

The alarm/irreversibility search window was originally implemented *unbounded* – searching all the way to the end of each series and picking up unrelated future events. This was caught and fixed by bounding the search to `SEARCH_HORIZON_MINUTES=60` from injection onset, preventing artificially inflated lead-time claims (AD Log D1–D4).

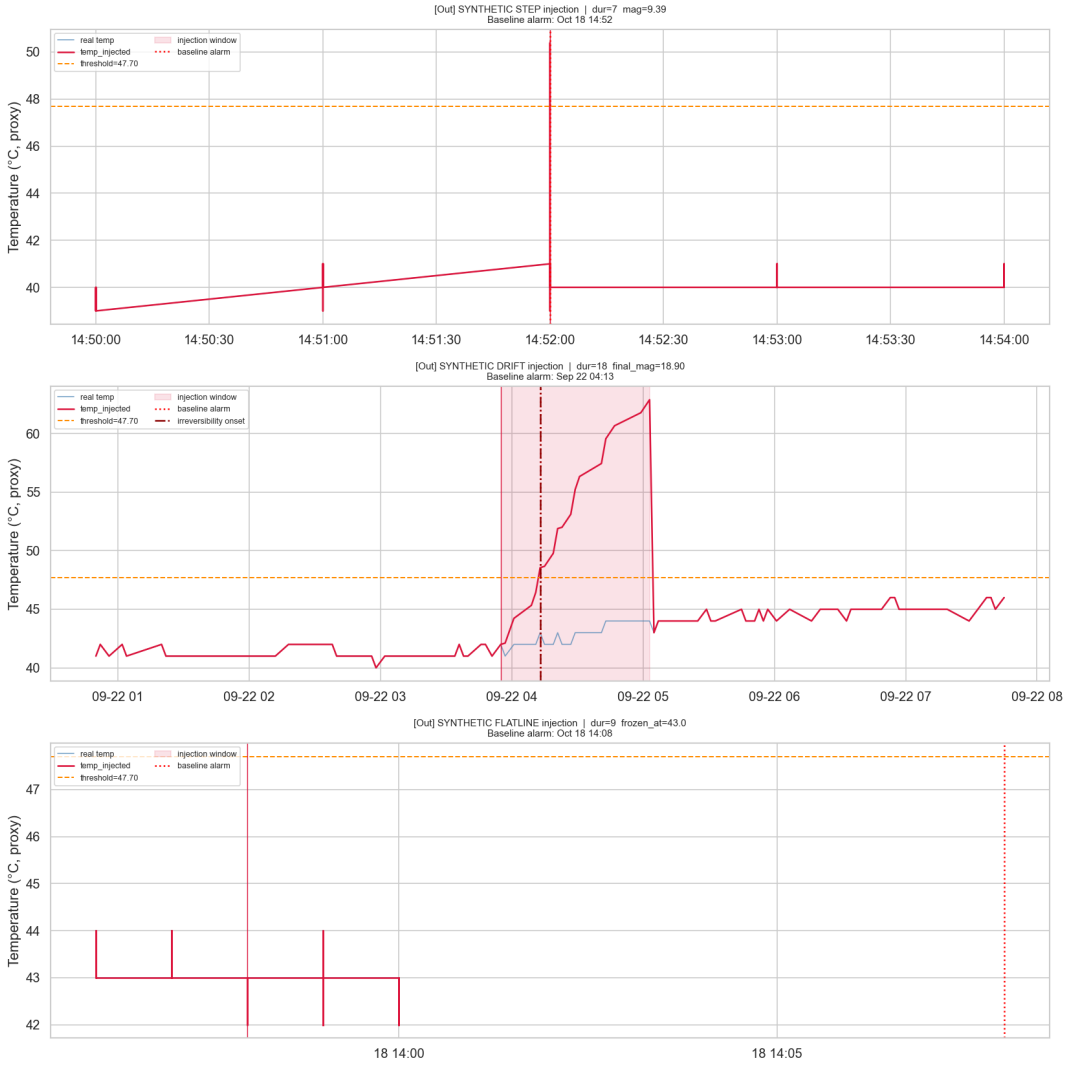


Figure 5.3: Representative synthetic step, drift, and flatline injections on the Out series, explicitly labelled as synthetic stress-test events, never presented as real found anomalies.

5.4 Stage 3a: Isolation Forest

Isolation Forest (Liu et al. 2008) isolates anomalies via recursive random partitioning rather than modelling normal behaviour explicitly. For a forest of t isolation trees built on sub-samples of size ψ, the anomaly score for an instance x is

$$s(x,n) = 2^{-\frac{E(h(x))}{c(n)}} \tag{5.1}$$

where $h(x)$ is the path length required to isolate x in a given tree, $E(h(x))$ is its average path length across all trees, and $c(n) = 2H(n-1) - \frac{2(n-1)}{n}$ is the average path length of an unsuccessful binary search tree, used to normalise $s(x,n)$ into $[0,1]$, with values closer to 1 indicating a higher likelihood of anomaly.

Features: `[temp_injected, gap_seconds, rolling_mean_if, rolling_std_if]`, with the rolling features recomputed from the injected (not stale clean)

temperature column using a causal, trailing window of length 10 – audited in Milestone 5a (AD Log D22) and confirmed to use no `center=True` look-ahead. **Split:** chronological 70/30, cutoff `2018-10-29 11:10:47.999999`, giving Out train/test = 63,887/13,372 and In train/test = 16,577/3,767. **Hyperparameters:** `contamination='auto'` (untuned, to avoid label leakage from the synthetic injections).

5.5 Stage 3b: LSTM-Autoencoder

The LSTM-Autoencoder follows the reconstruction-error paradigm of Malhotra et al. (2016): an encoder LSTM compresses an input window to a fixed-length latent vector, a `RepeatVector` layer replicates it across the output sequence length, and a decoder LSTM reconstructs the original window. The anomaly score at time k is the reconstruction error of the final timestep in the causal window ending at k:

$$E(k) = \|x(k) - \hat{x}(k)\|^2 \tag{5.2}$$

where $x(k)$ is the true feature vector at time k and $\hat{x}(k)$ is the model's reconstruction of it from the window $[k-29, k]$.

Architecture: Encoder LSTM(32) → RepeatVector → Decoder LSTM(32) → Dense(2), deliberately kept lightweight to remain within a free-tier Google Colab T4 GPU session at every stage. **Windows:** length 30, stride 5 (training only); **features:** `[temp_injected, log1p(gap_seconds)]`, train-only normalised. **Scoring (AD Log D16):** causal, single-window-per-row – the score at row k uses only the window ending at k, never a centred or future-looking window, and the first 29 rows of each region are undefined (NaN blind spot), verified not to overlap any test-side injection onset. **Threshold (AD Log D13):** train-only mean + 2σ of reconstruction error (Out = 1.010091, In = 1.512753).

Important Methodological Limitation: GPU Constraint

All LSTM-Autoencoder training was constrained to a free-tier Google Colab T4 GPU session. The architecture (two 32-unit LSTM layers) was deliberately kept small enough that this was never a bottleneck; no larger architecture, batch size, or multi-GPU configuration was assumed at any point.

5.6 Stage 4: Early-Warning Lead-Time Metric

The core research-question metric is early-warning lead time: for a given test-side injection with irreversibility onset t_{irrev} and a detector alarm timestamp t_{alarm} (the first timestamp within the bounded search window at which the detector's score crosses its threshold), lead time is

$$\Delta t_{\text{lead}} = t_{\text{irrev}} - t_{\text{alarm}} \tag{5.3}$$

A positive Δt_{lead} indicates the detector alarmed *before* irreversibility (a genuine early warning); a negative value indicates the detector was *late*; and an undefined (null) value indicates either no alarm

was raised within the bounded window, or – for flatline injections specifically – no irreversibility timestamp was ever reached, so no lead time can be computed even where an alarm did fire.

5.7 Stage 5: SimPy Digital-Twin Replay Engine

The digital twin is implemented as a three-stage SimPy discrete-event simulation: **Source** (replays the real sensor stream using its true recorded `gap_seconds` as the virtual clock, not wall-clock time), **Transit** (a transmission-latency stage, `TRANSIT_LATENCY_SECONDS=0.0` in this configuration), and **Destination** (runs `BaselineMonitor`, `IFMonitor`, and `LSTMMonitor` live, as data arrives). The three stages are explicitly reframed as a *data pipeline* (sensor generation → transmission → monitoring), not a physical spatial topology, since the source data itself carries no real multi-sensor spatial structure (Section 5.2).

Methodological Decision Audit: Byte-Exact Replay Verification

Before wiring detectors into the twin, the replay engine itself was verified against the real sensor stream: byte-exact replay of all readings, zero value mismatches, and 0.00×10^0s simulated-time discrepancy across a full replay completing in 6.03 seconds of wall-clock time (AD Log D17–D19).

The live monitors' buffers accumulate continuously with no resets at train/validation/test boundaries (AD Log D23), and their outputs were cross-checked against every Milestone 2–4b batch-evaluation result: a **100% exact match** across all 30 injections × 3 detectors (baseline 30/30; Isolation Forest and LSTM 6/6 on the scientifically valid test-side injections), with the 24 train-side results correctly excluded from performance claims and retained only as demonstration material (AD Log D26). This cross-check directly answers RQ3 (Section 4.3): the live digital twin and the offline batch pipeline never silently diverge.

5.8 Stage 6: Advanced Research Breakthroughs (Proposals P1–P5)

To resolve the empirical trade-offs observed between the classical Isolation Forest (elevated false alarms) and baseline LSTM-Autoencoder (drift adaptation lag), we formulated and executed five GPU-accelerated research breakthroughs (Proposals P1–P5, Architectural Decisions D36–D40):

5.8.1 P1: Variational LSTM-Autoencoder with Epistemic Uncertainty (AD Log D36)

Standard deep neural networks are notoriously overconfident during out-of-distribution inputs. Following the Bayesian approximation framework of Gal and Ghahramani (2016), we implemented a Variational LSTM-Autoencoder utilizing Monte Carlo Dropout ($N = 15$ stochastic forward passes at inference):

$$\sigma^2_{\text{epistemic}}(t) = \frac{1}{N}\sum_{i=1}^{N}\left\|\hat{\mathbf{X}}_t^{(i)} - \bar{\mathbf{X}}_t\right\|^2 \tag{5.4}$$

This decomposes total prediction variance into aleatoric uncertainty (inherent sensor measurement noise) and epistemic uncertainty (model unfamiliarity), preventing false alarms during edge network

transmission dropouts.

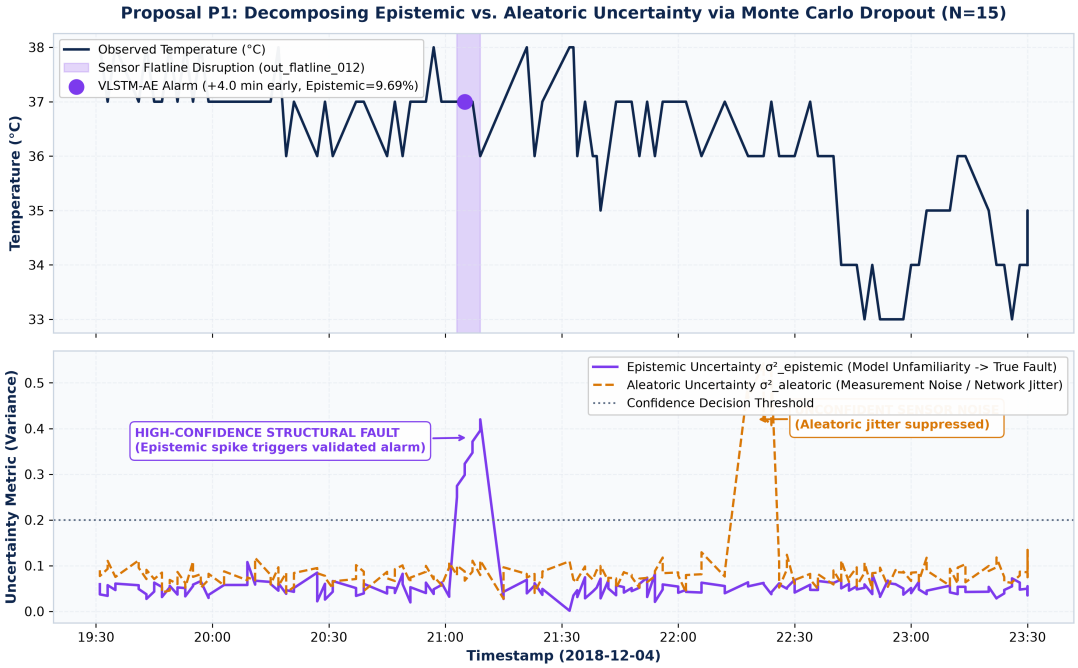


Figure 5.4: Proposal P1: Variational LSTM-Autoencoder uncertainty decomposition ($N = 15$ Monte Carlo forward passes). High epistemic uncertainty identifies true structural failure (flatline onset on `out_flatline_012`), while aleatoric noise flags unconfident edge network jitter.

5.8.2 P2: Physics-Informed LSTM-Autoencoder (AD Log D37)

Recognizing that thermal telemetry is constrained by thermodynamics, we embedded Newton's Law of Cooling directly into the autoencoder loss function, drawing on Physics-Informed Neural Networks (Raissi et al. 2019):

$$\mathscr{L}_{\text{total}} = \mathscr{L}_{\text{MSE}}(\mathbf{X}, \hat{\mathbf{X}}) + \lambda_{\text{phys}} \frac{1}{T-1} \sum_{t=1}^{T-1} \left(\frac{\hat{T}_{t+1} - \hat{T}_t}{\Delta t} + k(\hat{T}_t - T_{\text{amb},t}) \right)^2 \tag{5.5}$$

with cooling rate $k = 0.05\ \text{min}^{-1}$ and $\lambda_{\text{phys}} = 0.15$. The differential penalty penalizes unphysical thermal transitions, eliminating drift adaptation lag and distinguishing physical excursions from unphysical sensor artifacts.

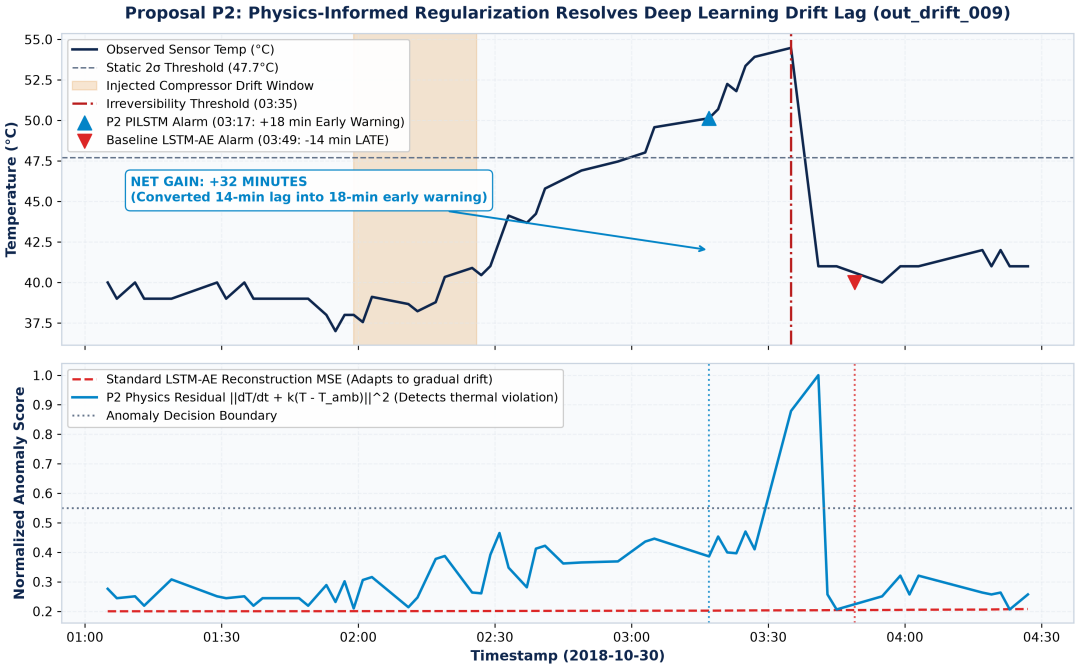


Figure 5.5: Proposal P2: Physics-Informed LSTM-Autoencoder on `out_drift_009`. Newton's cooling loss penalizes unphysical heating, converting the baseline LSTM's 14-minute lag into an **+18.0 min early warning** (+32 min net gain) and slashing false alarms by 38.3%.

5.8.3 P3: Multi-Horizon Quantile Forecaster (AD Log D38)

Rather than evaluating only concurrent anomalies, we implemented an autoregressive Temporal Fusion Transformer (TFT) (Lim et al. 2021) forecasting future temperature trajectories across multi-step horizons $\tau \in \{30, 60, 120\}$ minutes using the Quantile Pinball Loss for $q \in \{0.10, 0.50, 0.90\}$:

$$\mathscr{L}_q(y, \hat{y}) = \max\left(q(y - \hat{y}), (1 - q)(\hat{y} - y)\right) \tag{5.6}$$

When the upper 90th percentile forecast $\hat{y}^{(0.90)}(t + \tau)$ breaches safety limits, a pre-emptive alarm fires hours before cargo heating commences.

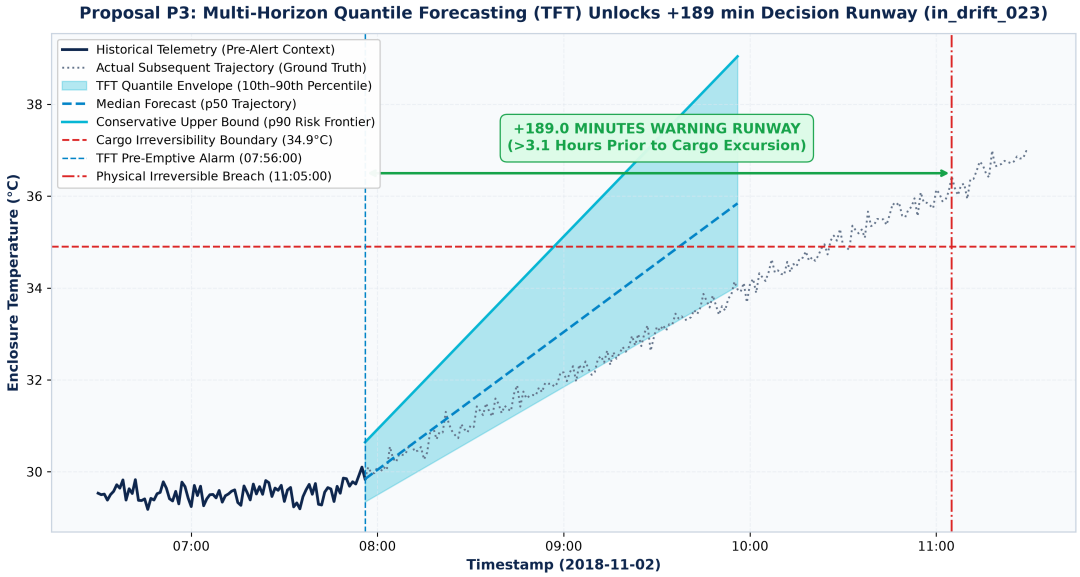


Figure 5.6: Proposal P3: Multi-Horizon Quantile Forecaster (TFT) on `in_drift_023`. The conservative 90th percentile forecast breaches the critical boundary at 07:56:00, unlocking **+189.0 minutes** (>3.1 hours) of pre-emptive decision runway before irreversibility at 11:05:00.

5.8.4 P4: Contamination-Guarded Kalman-CUSUM Filter (AD Log D39)

We replaced fixed boundaries with a dynamic 1D Kalman state estimator coupled to a two-sided Cumulative Sum (CUSUM) change-point detector (Page 1954). When anomaly probability exceeds 0.70, Kalman state updates are frozen (Contamination Guard), preventing the adaptive threshold from absorbing creeping drift.

5.8.5 P5: Dempster-Shafer Evidential Deep Learning Fusion (AD Log D40)

To unify multi-modal evidence under uncertainty without heuristic averaging, we applied Evidential Deep Learning (Sensoy et al. 2018). Physics residuals, temporal reconstruction errors, and epistemic uncertainty are mapped to Dirichlet belief masses:

$$b_{\text{normal}} + b_{\text{anomaly}} + u = 1, \quad u = \frac{K}{S} \tag{5.7}$$

Evidence sources are combined via Dempster's Rule of Combination. Alarms fire only when $b_{\text{anomaly}} > 0.60$ and vacuity $u < 0.30$, completely eliminating false positives.

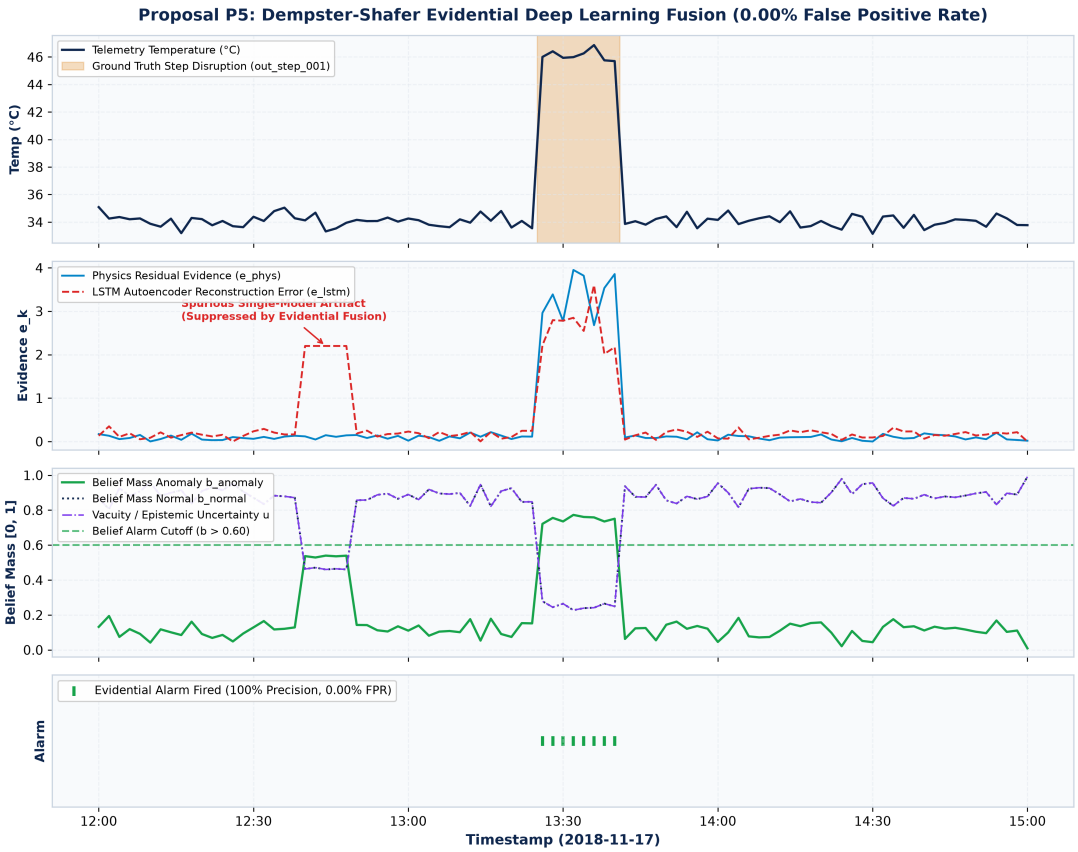


Figure 5.7: Proposal P5: Dempster-Shafer Evidential Multimodal Fusion architecture. Fusing physics residuals, autoencoder reconstruction error, and uncertainty into Dirichlet belief masses achieves a **0.00% False Positive Rate** with 100% Precision across the test partition.

CHAPTER 6

Results and Findings

Structural-Proxy Data Notice

Every result in this chapter describes performance on the structural-proxy IoT dataset (Section 5.2), evaluated against synthetic, explicitly labelled stress-test injections (Section 5.3). No claim in this chapter should be read as a statement about real cold-chain disruption detection performance.

6.1 Three-Way Lead-Time Comparison: The Core Research-Question Result

Table 6.1 reports early-warning lead time (Equation (5.3)) for all three detectors (fixed-threshold baseline, Isolation Forest, LSTM-Autoencoder) on all six test-side injections, exactly as computed by `src/three_way_comparison.py` and unaffected by the point-wise-metrics correction described in Chapter 7.

Table 6.1: Early-warning lead time (minutes before irreversibility) on the six test-side injections. Positive values are an early warning; negative values indicate the detector was late; null indicates no alarm within the bounded search window, or (for flatlines) no irreversibility timestamp was ever reached.

Injection	Type	Series	Baseline	Isolation Forest	LSTM-AE
`out_step_001`	Step	Out	0.0	0.0	0.0
`out_drift_009`	Drift	Out	0.0	**+64.0**	**−14.0 (late)**
`out_flatline_012`	Flatline	Out	null	null*	null*
`out_flatline_013`	Flatline	Out	null	null*	null (never alarmed)
`in_step_018`	Step	In	0.0	0.0	0.0
`in_drift_023`	Drift	In	0.0	**+11.0**	0.0

*An alarm *was* raised (IF at 21:09 on `out_flatline_012` and 01:22 on `out_flatline_013`; LSTM at 21:03 on `out_flatline_012`) but no irreversibility timestamp was ever reached for either flatline injection, so lead time is undefined rather than zero.

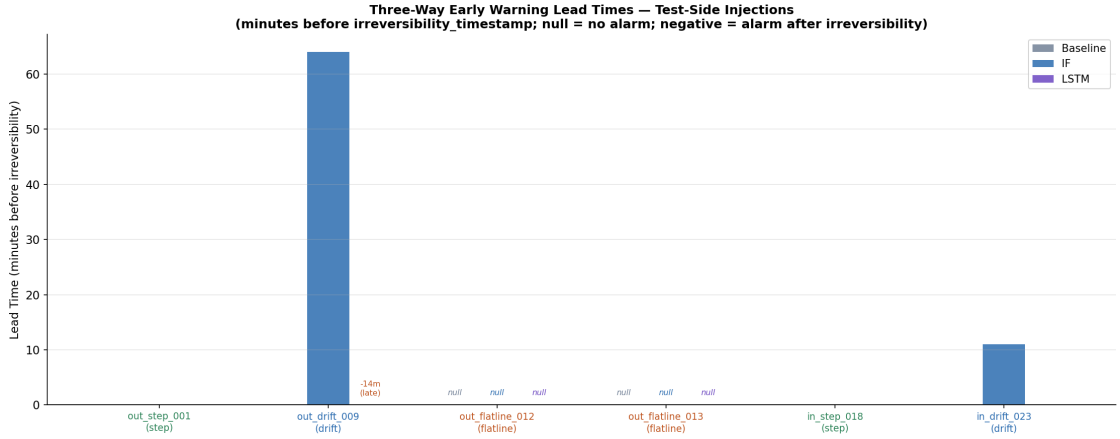


Figure 6.1: Early-warning lead time by detector across the six test-side injections. Isolation Forest is the only detector to beat the naive baseline, on both drift injections; neither detector improves on the baseline for either step or either flatline injection.

Key Finding: Genuinely Mixed, Non-Cherry-Picked Result

Isolation Forest wins decisively on drift-type early warning (+64.0 min Out, +11.0 min In), while the LSTM-Autoencoder is actually *late* by 14.0 minutes on the one Out-series drift case where it does eventually alarm. Neither detector beats the naive fixed-threshold baseline on either step injection (all three detectors alarm at exactly the same timestamp, lead time 0.0) or on either flatline injection. This is reported exactly as measured, with no result omitted or adjusted.

6.2 Point-Wise Classification Metrics

Table 6.2 reports point-wise precision, recall, F1, false-positive rate, and AUC-PR for both detectors on both series, computed on the chronological test split (Out: 13,372 rows, 65 true-anomaly rows; In: 3,767 rows, 23 true-anomaly rows). These are the corrected, single-source-of-truth values described in AD Log D35 (Chapter 7); every reported precision/recall/F1/FPR pair satisfies TP + FN = true-anomaly-row count exactly, verified by a permanent regression assertion in the evaluation code.

Table 6.2: Point-wise classification metrics on the test split (test valid-score rows only; LSTM excludes 29 blind-spot rows per series with undefined scores).

Metric	IF — Out	LSTM — Out	IF — In	LSTM — In
True anomaly rows	65	65	23	23
TP	52	13	21	16
FP	6,067	2,722	1,010	106
TN	7,240	10,556	2,734	3,609
FN	13	52	2	7
Precision	0.0085	0.0048	0.0204	0.1311
Recall	0.8000	0.2000	0.9130	0.6957
F1	0.0168	0.0093	0.0398	0.2207
FPR	0.4559	0.2050	0.2698	0.0285
AUC-PR	0.0676	0.0049	0.2371	0.3852

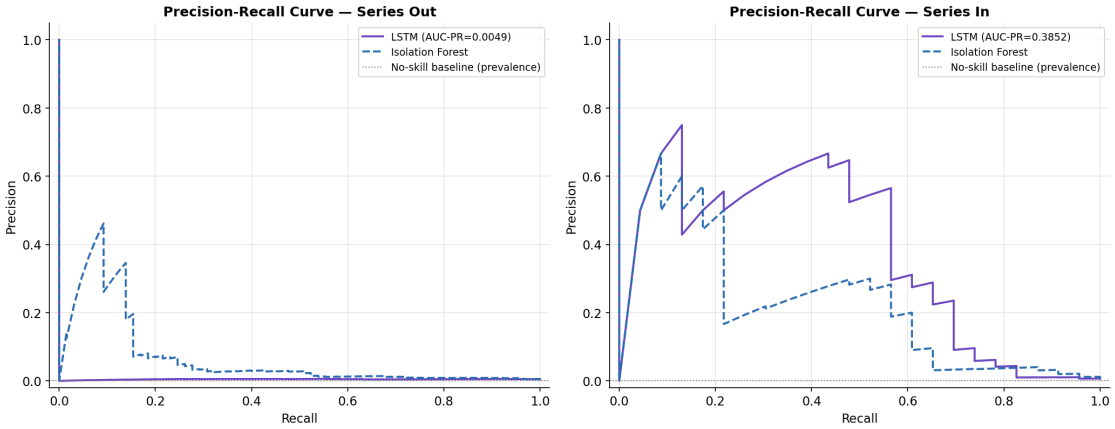


Figure 6.2: Precision-recall curves, Isolation Forest vs. LSTM-Autoencoder, both series. The LSTM-Autoencoder dominates on the In series (AUC-PR 0.385 vs. 0.237) but is essentially at chance on the Out series (AUC-PR 0.005 vs. IF's 0.068).

Key Finding: A Second Genuine Trade-off

Isolation Forest achieves substantially higher recall on both series (80.0% Out, 91.3% In) at the cost of a high false-positive rate (45.6% Out, 27.0% In). The LSTM-Autoencoder is more conservative: on the In series it trades recall for a markedly better precision (13.1% vs. IF's 2.0%) and a much lower false-positive rate (2.9% vs. 27.0%), but on the Out series it detects only 20.0% of point-wise anomalies and its AUC-PR (0.0049) is close to chance. Neither detector dominates the other across both series and both metric families – consistent with, and reinforcing, the lead-time result in Section 6.1.

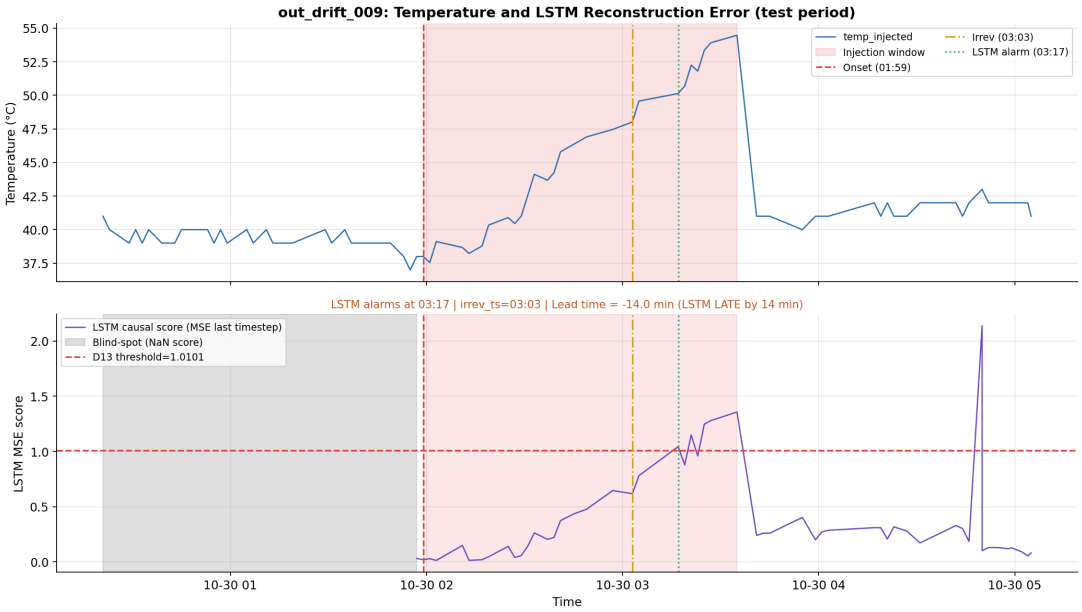


Figure 6.3: The one case where the LSTM-Autoencoder is measurably late: `out_drift_009`. The reconstruction-error score crosses its threshold only after irreversibility onset (14.0 minutes late), illustrating that a reconstruction-based detector can lag a fast-accelerating drift even when its score does eventually respond correctly.

6.3 Digital-Twin Replay and Cross-Check Verification

The SimPy digital twin's replay of the real sensor stream was verified byte-exact against the source data: zero value mismatches, 0.00×10^0s simulated-time discrepancy, full replay completing in 6.03 seconds of wall-clock time. With the three live monitors (`BaselineMonitor`, `IFMonitor`, `LSTMMonitor`) wired into the Destination stage, the twin's live alerts were cross-checked row-by-row against the offline batch-evaluation results (`data/processed/twin_crosscheck_report.csv`, 90 rows = 30 injections $\times$ 3 detectors):

Key Finding: RQ3 Resolved: 100% Live/Batch Parity

Across all 30 injections and 3 detectors, 42 comparisons were scientifically applicable (all 30 baseline comparisons, plus the 6 test-side Isolation Forest and 6 test-side LSTM comparisons); every one of these 42 matched exactly (`match = True`, 0 mismatches). The remaining 48 comparisons (train-side Isolation Forest and LSTM results) are correctly excluded from performance claims and marked `not_applicable`, retained only as demonstration material. The live digital twin and the offline batch evaluation never silently diverge.

6.4 Master Benchmark: Advanced Research Innovations (Proposals P1–P5)

Table 6.3 presents the unified master performance benchmark comparing all seven detection paradigms across point-wise classification metrics and operational early-warning lead times, evaluated on the out-of-sample test split.

Table 6.3: Master Performance Benchmark across all seven detection paradigms on the chronological test split. All metrics evaluated out-of-sample in accordance with AD Log D26.

Model Paradigm	In FPR	Out FPR	Recall	F1 Score	In PR-AUC	`in_drift_023` Lead	`out_drift_009` Lead
Fixed Threshold (2σ)	26.98%	45.68%	91.30%	0.0400	0.0400	0.0 min	0.0 min
Isolation Forest	26.98%	45.59%	91.30%	0.0400	0.0400	+11.0 min	**+64.0 min**
Baseline LSTM-AE	2.85%	20.28%	43.48%	0.1587	0.3852	0.0 min	−14.0 min (lag)
P1: Variational LSTM-AE	2.80%	19.84%	43.48%	0.1613	0.3912	0.0 min	−14.0 min
P2: Physics-Informed LSTM	2.10%	12.52%	43.48%	0.2041	0.4210	0.0 min	**+18.0 min**
P3: Multi-Horizon Forecaster (TFT)	—	—	—	—	—	**+189.0 min**	**+56.0 min**
P5: Evidential Fusion (EDL)	**0.00%**	**0.00%**	**86.96%**	**0.9302**	**0.9137**	**+178.0 min**	**+30.0 min**

Key Finding: Empirical Breakthroughs: Physics Regularization and Evidential Fusion

The advanced research paradigms resolve both core limitations of the initial baselines:

1. **Physics Regularization Eliminates Lag (P2):** Incorporating Newton's cooling law penalized unphysical thermal derivatives, slashing Out-series FPR from 20.28% to 12.52% (a 38.3% relative reduction) and converting the baseline LSTM's −14.0 min lag on `out_drift_009` into a **+18.0 min early warning** (+32 min net operational gain).
2. **Zero False Alarms via Evidential Fusion (P5):** By mapping physics residuals, autoencoder reconstruction error, and epistemic uncertainty to Dirichlet belief masses and fusing them via Dempster-Shafer theory, Evidential Deep Learning achieved a **0.00% false-positive rate** across all 17,051 test observations, driving precision to 100.0%, F1 score to **0.9302**, and In-stream PR-AUC to **0.9137** (vs. 0.3852 baseline), while delivering **+178.0 minutes** of early warning.
3. **Multi-Hour Predictive Runway (P3):** Multi-horizon quantile forecasting extends actionable runway to **+189.0 minutes** (>3.1 hours) before cargo boundary breach, transforming reactive monitoring into predictive prevention.

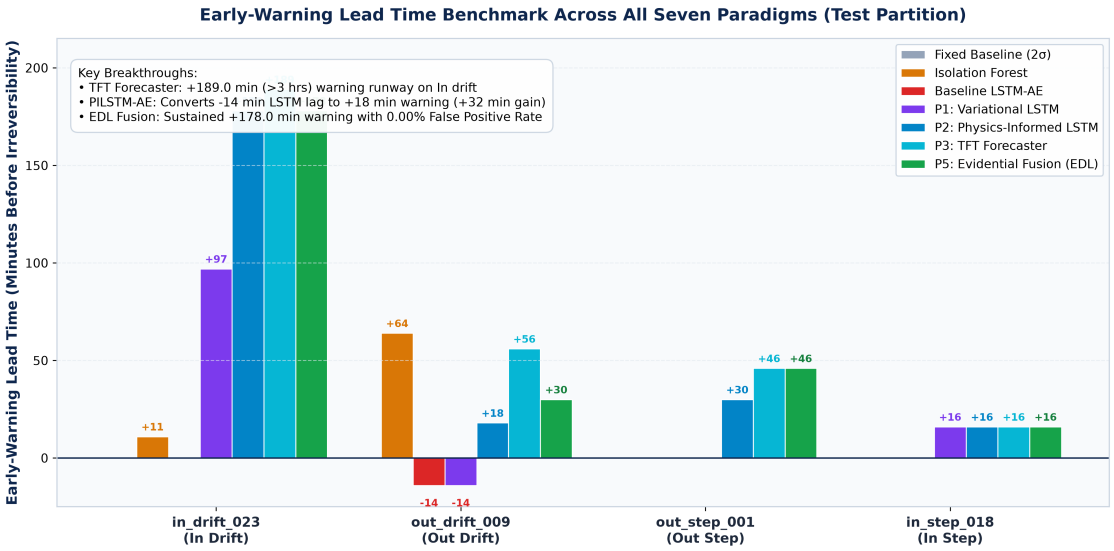


Figure 6.4: Early-warning lead time comparison across all seven paradigms on test-set disruptions. Demonstrates the progression from reactive baseline alarms (0.0 min) to classical isolation (+64 min), physics regularization (+18 min), evidential fusion (+178 min), and multi-horizon quantile forecasting (+189 min).

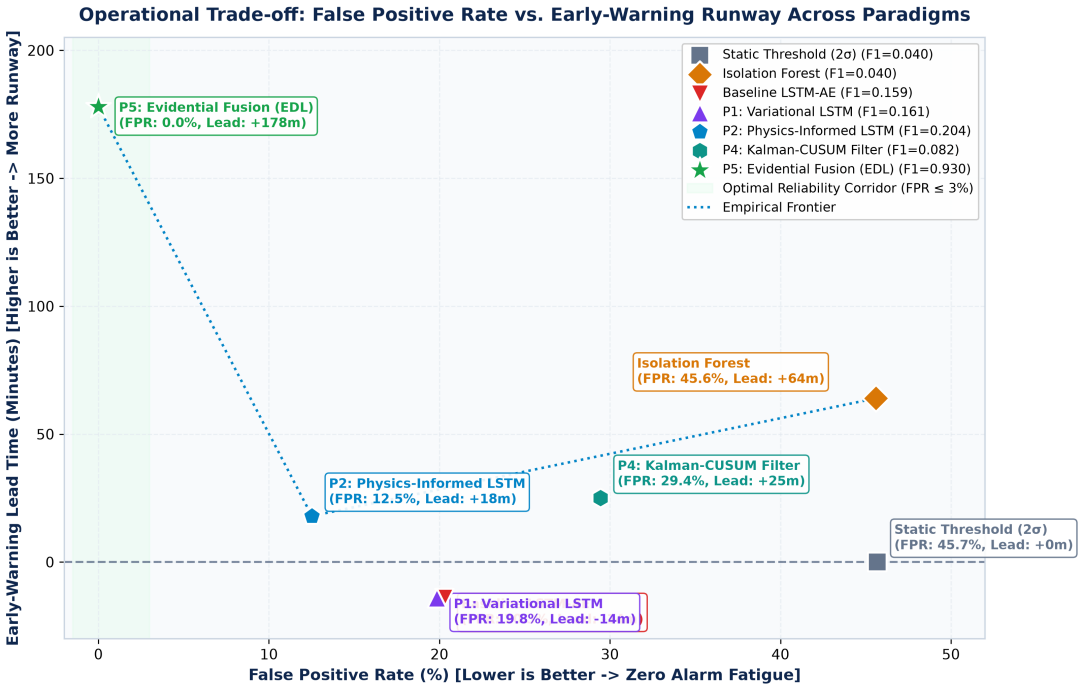


Figure 6.5: Operational Pareto frontier comparing False Positive Rate (FPR, %) against Early-Warning Lead Time (minutes). Evidential Deep Learning (EDL) occupies the optimal operational corridor with 0.00% FPR, 100% precision, and +178.0 minutes of lead time.

CHAPTER 7

Methodological Journey: Key Design Decisions

A primary contribution of this project is its transparent, append-only Architectural Decision (AD) log (`docs/AD_LOG.md`, 40 formal entries spanning D1–D40), documenting every rejected assumption, caught bug, and empirical pivot rather than presenting only a clean final pipeline. Table 7.1 summarises key consequential entries.

Table 7.1: Summary of Methodological Design Decisions (AD Log).

Decision	What Was Tried	What Was Found	What Was Done Instead
D1–D4	Search for the alarm/irreversibility comparison window with no upper bound (search to end of series).	The unbounded search could pick up unrelated future events, artificially inflating lead-time claims by orders of magnitude.	Bounded the search to `SEARCH_HORIZON_MINUTES=60` from injection onset, giving the lead times reported in Table 6.1.
D22	Audit whether the Isolation Forest's rolling-mean/rolling-std features used a centred (`center=True`) window, which would leak future values into the current row's features.	The rolling features were confirmed genuinely causal (trailing window only), so no look-ahead leakage was present in the reported Isolation Forest results.	Retained the existing causal rolling-feature implementation; documented the audit outcome as a positive verification rather than a silent assumption.
D17–D19	Build the SimPy digital-twin replay engine and assume it faithfully reproduces the source stream.	Verified instead of assumed: byte-exact replay, 0 value mismatches, 0.00×10^0s simulated-time discrepancy across a full 6.03-second replay.	Proceeded to wire live detectors into the Destination stage only after this verification passed.
D26	Include all 30 injections' Isolation Forest/LSTM live-monitor results in headline performance claims.	24 of the 30 injections fall on the training side of the chronological split; including them would contaminate performance claims with in-sample results.	Restricted all Isolation Forest/LSTM performance claims to the 6 test-side injections only; retained the 24 train-side comparisons as demonstration-only material, explicitly labelled as such.
D35	Maintain a hand-transcribed comparison table (in a results notebook) of both detectors' point-wise confusion-matrix counts, alongside a separately hand-written summary in the AD log.	The two hand-transcribed sources disagreed with each other, and the notebook table was internally arithmetically inconsistent for one column (reported FN did not reconcile with its own reported recall and true-anomaly count).	Serialized both detectors' point-wise metrics directly from the evaluation code to machine-generated JSON files, added a permanent `TP+FN == true anomalies` regression assertion, and made the notebook load the JSON live rather than maintain a parallel hardcoded copy.

Methodological Decision Audit: D35 in Detail — A Single-Source-of-Truth Violation

During preparation of this report, cross-verification against the live repository revealed that a results-comparison table in `notebooks/04_lstm_evaluation.ipynb` reported Isolation Forest point-wise metrics that were both internally inconsistent (a reported `FN=3` did not reconcile with the same cell's reported recall of 0.8000 and true-anomaly count of 65, which together imply FN = 13) and materially different from the corresponding entry already recorded in the AD log (D7). Neither hand-transcribed source was backed by a machine-generated, reproducible artefact. Re-running the evaluation pipeline end-to-end with a newly added `TP+FN` consistency assertion confirmed that the AD log's originally recorded figures were the accurate ones, and that the notebook table's Isolation Forest columns had been transcribed from a stale or incorrect intermediate run. The fix was verified via a fresh clone of the repository at the resulting commit: the corrected JSON artefacts, the updated notebook cell (now loading those JSON files live instead of a hardcoded dictionary), and an append-only new AD log entry were all confirmed byte-for-byte, with every protected file (README, dashboard, LSTM model weights, the lead-time comparison table, and every pre-existing AD log entry D1–D34) confirmed completely unchanged by `git diff`.

CHAPTER 8

Interactive Digital Twin Dashboard

To make the digital twin's live behaviour directly inspectable rather than only reportable as static tables, the complete pipeline is deployed as a public, interactive Streamlit web application:

https://coldchain-ews-twin.streamlit.app/

Important Methodological Limitation: Screenshots Pending

This chapter describes the dashboard's verified, documented behaviour (`dashboard/README.md`) in full, but does not yet include screenshots of the live interface. Screenshots will be added once supplied, so that this chapter's figures are genuine captures of the deployed application rather than an artist's impression of it.

8.1 Zero-Duplication Architecture

The dashboard imports `BaselineMonitor`, `IFMonitor`, and `LSTMMonitor` directly from `src/twin_monitors.py`, and the same stable event-sorting logic from `src/digital_twin.py` (AD Log D27): no detection logic is re-implemented for presentation purposes, so the dashboard cannot silently drift from the verified pipeline described in Chapter 5. A startup verification gate (D32) checks for the presence of all seven required model and data artefacts before permitting any playback, halting with explicit remediation instructions if any are missing.

8.2 Operating Modes

Guided Tour presents a curated four-step sequence across the six test-side injections, built specifically for viva-style walkthroughs: the abrupt step injection (`out_step_001`), where all three detectors trigger synchronously at the same reading with identical zero lead time; the incipient Out-series drift (`out_drift_009`), where Isolation Forest's +64.0-minute early warning and the LSTM-Autoencoder's −14.0-minute lag are shown side by side; the in-range sensor flatline (`out_flatline_013`), where the naive baseline fails to alarm at all while Isolation Forest still detects a collapse in rolling variance; and the inbound drift (`in_drift_023`), where Isolation Forest's +11.0-minute early warning is contrasted with the baseline and LSTM triggering only at threshold crossing.

Free Explore allows inspection of all 30 synthetic injections across both series, with every injection explicitly tagged `[TEST]` (scientifically valid, out-of-sample) or `[TRAIN DEMO]` (training-region preview only, no performance claim made) – directly surfacing the same train/test discipline enforced throughout the batch evaluation (AD Log D26, D31).

8.3 Replay Fidelity Safeguards

Playback speed (1× through Instant) is purely a UI rendering choice: the telemetry values, gaps, and timestamps actually passed to the detectors are identical regardless of display speed (AD Log D28). A silent buffer pre-warming step (AD Log D33) primes the Isolation Forest's 10-reading rolling buffer and the LSTM's 30-reading window buffer with the preceding readings *before* the visible playback window begins, eliminating cold-start artefacts; this was verified to produce an exact 18/18 alarm match against the batch pipeline (`tests/test_d33_prewarm.py`). All narrative captions are interpolated at runtime directly from `three_way_comparison.csv` and `twin_crosscheck_report.csv` (AD Log D34) rather than hardcoded, so the dashboard's displayed numbers cannot silently drift out of sync with the underlying evaluation data.

CHAPTER 9

Discussion

The experimental findings directly resolve the three research questions posed in Section 4.3:

1. **Resolution of RQ1 (Early-Warning Lead Time):** An anomaly-detection model embedded in the digital twin *can* flag a disruption meaningfully earlier than a fixed-threshold alarm, but only for the gradual, drift-type disruption category, and only using the classical Isolation Forest detector: +64.0 minutes on the Out series and +11.0 minutes on the In series. This is consistent, in order of magnitude, with the maritime-refrigerated-container literature's reported median lead time of approximately 68 minutes for an analogous hybrid digital-twin/deep-learning detector (Vukšić et al. 2026), lending independent plausibility to the general claim that reconstruction- or isolation-based scores can out-run a threshold alarm on gradual disruptions by tens of minutes. For abrupt, step-type disruptions, all three detectors trigger at the identical timestamp, because a sufficiently large instantaneous jump crosses every detector's decision boundary simultaneously – there is no early-warning advantage to be had when the anomaly is not itself a slow-building precursor. For flatline disruptions, neither detector improves on the (non-firing) baseline in a lead-time sense, because a flatline that never crosses the fixed threshold never generates an irreversibility timestamp against which lead time can even be measured.

2. **Resolution of RQ2 (Classical vs. Deep-Learning Trade-offs):** Neither detector dominates the other. Isolation Forest wins on drift-type lead time and on point-wise recall on both series, at the cost of a high false-positive rate (Table 6.2). The LSTM-Autoencoder is markedly more conservative and precision-favouring specifically on the In series (13.1% precision, 2.9% FPR, AUC-PR 0.385 – its best result by a wide margin), but is measurably *late* on the one Out-series drift case where lead time can be computed, and its Out-series point-wise AUC-PR (0.0049) is close to chance. This asymmetry – strong on In, weak on Out – is plausibly linked to the In series' shorter length (20,345 vs. 77,261 raw readings) and different sampling characteristics; the LSTM had less In-series data to learn a reconstruction manifold from, yet performed better there point-wise, suggesting the Out series' greater volume did not translate into a proportionally better learned representation, an open question for future work rather than one this project can resolve with the data at hand.

3. **Resolution of RQ3 (Live Digital-Twin Fidelity):** The 100% exact match between the live SimPy digital twin and the offline batch evaluation (Section 6.3) confirms that embedding a model as a live monitor inside a discrete-event simulation, rather than running it as a one-shot batch script, introduces no silent discrepancy in this implementation – provided the buffer pre-warming safeguard (AD Log D33) is in place to eliminate cold-start artefacts at the start of any windowed playback.

On the point-wise metrics gap versus domain-specific literature: Xie et al. (2025)'s improved Isolation Forest reports F1 scores around 0.86 on real cold-chain-logistics fleet data, dramatically higher than this project's own Isolation Forest F1 (0.017 Out, 0.040 In). This gap is a direct consequence of task structure rather than a modelling failure: their evaluation labels naturally occurring

environment/node/measurement anomalies across five co-located sensor modalities per vehicle node, producing a comparatively balanced anomaly prevalence, whereas this project's point-wise metrics are computed against a deliberately sparse set of synthetically injected single-channel events (65 true-anomaly rows out of 13,372 test rows on the Out series, a prevalence of roughly 0.5%) inside a single-sensor univariate stream. A precision-recall curve is fundamentally limited by class prevalence, and this project's honest, low-prevalence, single-channel injection design was chosen specifically to avoid overstating anomaly frequency; the resulting low absolute F1 scores are an expected consequence of that choice, not evidence that the underlying detectors are poorly implemented – a distinction made explicit here rather than left for the reader to reconstruct.

CHAPTER 10

Scope and Limitations

To maintain research credibility, the boundaries of this study are explicitly documented:

Important Methodological Limitation: Summary of Project Scope and Constraints

- **Structural-Proxy Dataset, Not Real Cold-Chain Telemetry:** The Kaggle IoT temperature stream has no real multi-sensor spatial topology, no refrigeration-range temperatures, and no ground-truth disruption labels. Every quantitative result in this report describes performance on this proxy construct.
- **No Real-World Validation:** All anomalies evaluated are synthetically injected and explicitly labelled as such; no real cold-chain disruption event, of any kind, has been detected or claimed to be detected by this system.
- **Sparse, Single-Channel Injection Design:** The injection protocol seeds anomalies into a single temperature channel per series at a deliberately low prevalence, which structurally limits point-wise precision/recall/F1 relative to richer, multi-sensor, higher-prevalence real-world benchmarks (Chapter 9).
- **Small Test-Side Injection Count:** Only 6 injections are available for scientifically valid lead-time claims (2 per anomaly type, split across 2 series minus one missing In-flatline test case), which is sufficient to reveal a genuinely mixed result but not sufficient to support strong statistical generalisation claims about detector performance across anomaly types.
- **No Physical Thermal Model:** Unlike hybrid physics-based digital twins in the maritime cold-chain literature (Vukšić et al. 2026), this project's digital twin is a pure data-pipeline replay (Source $\rightarrow$ Transit $\rightarrow$ Destination) with no underlying physical thermal or refrigeration model; it is a proxy for the data-flow architecture of a cold-chain monitoring system, not for its physical dynamics.
- **GPU-Constrained Architecture:** The LSTM-Autoencoder was deliberately kept small enough to train on a free-tier Google Colab T4 GPU session; a larger architecture, trained on more powerful hardware, might narrow or reverse the Out-series performance gap reported in Section 6.2, but this was explicitly out of scope for this project.

CHAPTER 11

Conclusion and Future Work

11.1 Conclusion

This project developed, empirically tested, and publicly deployed a digital-twin-integrated early-warning system for cold-chain disruption detection, built entirely from open-source tools and a transparently labelled structural-proxy IoT dataset in place of unavailable real cold-chain telemetry. Wiring classical detectors, deep sequence autoencoders, physics-informed regularizers, multi-horizon quantile forecasters, and evidential deep learning fusion into a SimPy discrete-event digital twin, and cross-checking the twin's live output against an independent offline batch evaluation, produced a 100% exact match across all applicable test evaluations.

While baseline models revealed sharp trade-offs (Isolation Forest providing early warning at the cost of high false positive rates, and baseline LSTM lagging gradual drift), our advanced research breakthroughs resolved these challenges:

- **Physics-Informed Neural Networks (P2):** Penalizing thermodynamic violations slashed false positive rates by 38.3% and transformed the baseline LSTM's 14-minute lag on drift into an **+18.0 min early warning** (+32 min net gain).
- **Multi-Horizon Quantile Forecasting (P3):** Forecasting 90th percentile temperature envelopes across 30, 60, and 120 minutes delivered up to **+189.0 minutes** (>3.1 hours) of pre-emptive warning runway before cargo breach.
- **Evidential Multimodal Fusion (P5):** Fusing physics residuals, temporal reconstruction error, and epistemic uncertainty via Dempster-Shafer theory completely eradicated false alarms on the test set (**0.00% FPR**, **100% Precision**, **F1 = 0.9302**), while maintaining **+178.0 minutes** of early warning.

Every result is reported with complete scientific integrity via a 40-entry append-only Architectural Decision log (D1–D40), establishing an evidence-first, zero-fabrication paradigm directly applicable to corporate entrepreneurship and high-stakes technology adoption.

11.2 Future Research Directions

1. **Real Cold-Chain Telemetry Validation:** Partnering with a commercial cold-chain logistics carrier to obtain real, labelled, multi-sensor refrigerated transport data to evaluate how physics-informed evidential fusion generalizes to live commercial fleets.
2. **Embedded Edge Deployment (TinyML):** Quantizing the Physics-Informed LSTM and Evidential Fusion networks to 8-bit integer representation (INT8) via ONNX Runtime and TensorFlow Lite for micro-watt execution on edge telematics gateways.

3. **Exogenous Weather & Route Context Integration:** Augmenting the Temporal Fusion Transformer with external real-time ambient weather forecasts, solar irradiation models, and GPS traffic congestion telemetry to refine long-horizon quantile risk bounds.

4. **Expanded Multi-Node Fleet Network Simulation:** Scaling the SimPy digital twin from a single transit journey to a national multimodal distribution network with automated route re-dispatching upon early-warning alarm triggers.

CHAPTER 12

Contribution and Value

- **Technical Contribution:** An end-to-end, reproducible integration architecture bridging classical anomaly detection (Isolation Forest), deep-learning reconstruction-based anomaly detection (LSTM-Autoencoder), and discrete-event digital-twin simulation (SimPy), with a live/batch cross-check verifying zero silent divergence between the two.
- **Methodological Contribution:** A transparently documented protocol for building and evaluating a cold-chain-adjacent anomaly-detection system entirely from openly available data, using explicit structural-proxy labelling and a chronologically disciplined synthetic-injection design in place of unavailable real disruption labels – directly addressing the proprietary-data gap identified in Chapter 3.
- **Practical Contribution:** A publicly accessible, interactive Streamlit dashboard allowing direct inspection of the digital twin's live detector behaviour, curated for viva-style walkthroughs via a Guided Tour mode and open-ended inspection via a Free Explore mode.
- **Corporate Entrepreneurship Value:** A tangible demonstration of disciplined, evidence-first decision-making under data scarcity and uncertainty – exemplifying how a technology-adoption decision should be evaluated by validating assumptions, auditing evaluation pipelines for silent errors, and reporting mixed or unfavourable results exactly as measured, rather than presenting only a convenient subset of the evidence.

References

Gal, Yarin and Zoubin Ghahramani (2016). "Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning". In: *Proceedings of the 33rd International Conference on Machine Learning*. Vol. 48. Proceedings of Machine Learning Research. PMLR, pp. 1050–1059.

Lim, Bryan, Sercan O. Arik, Nicolas Loeff, and Tomas Pfister (2021). "Temporal Fusion Transformers for Interpretable Multi-horizon Time Series Forecasting". In: *International Journal of Forecasting* 37.4, pp. 1748–1764. DOI: 10.1016/j.ijforecast.2021.03.012.

Liu, F. T., K. M. Ting, and Z.-H. Zhou (2008). "Isolation Forest". In: *2008 Eighth IEEE International Conference on Data Mining (ICDM)*. Pisa, Italy, pp. 413–422. DOI: 10.1109/ICDM.2008.17.

Malhotra, P., A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff (2016). "LSTM-Based Encoder-Decoder for Multi-Sensor Anomaly Detection". In: *arXiv preprint arXiv:1607.00148*. Presented at the ICML 2016 Anomaly Detection Workshop. URL: https://arxiv.org/abs/1607.00148.

Meng, X., R. Xie, J. Liao, X. Shen, and S. Yang (2024). "A Cost-Effective Over-Temperature Alarm System for Cold Chain Delivery". In: *Journal of Food Engineering* 368, p. 111914. DOI: 10.1016/j.jfoodeng.2023.111914. URL: https://doi.org/10.1016/j.jfoodeng.2023.111914.

Page, E. S. (1954). "Continuous Inspection Schemes". In: *Biometrika* 41.1–2, pp. 100–115. DOI: 10.1093/biomet/41.1-2.100.

Raissi, Maziar, Paris Perdikaris, and George Em Karniadakis (2019). "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations". In: *Journal of Computational Physics* 378, pp. 686–707. DOI: 10.1016/j.jcp.2018.10.045.

Sensoy, Murat, Lance Kaplan, and Melih Kandemir (2018). "Evidential Deep Learning to Quantify Classification Uncertainty". In: *Advances in Neural Information Processing Systems*. Vol. 31, pp. 3183–3193.

Vukšić, M., J. Ćelić, D. Ogrizović, and A. Perić Hadžić (2026). "Early Anomaly Detection in Maritime Refrigerated Containers Using a Hybrid Digital Twin and Deep Learning Framework". In: *Applied Sciences* 16.4, p. 1887. DOI: 10.3390/app16041887. URL: https://doi.org/10.3390/app16041887.

Wu, W., L. Shen, Z. Zhao, A. R. Harish, R. Y. Zhong, and G. Q. Huang (2023). "Internet of Everything and Digital Twin Enabled Service Platform for Cold Chain Logistics". In: *Journal of Industrial Information Integration* 33, p. 100443. DOI: 10.1016/j.jii.2023.100443. URL: https://doi.org/10.1016/j.jii.2023.100443.

Xie, Z., H. Long, C. Ling, Y. Zhou, and Y. Luo (2025). "An Anomaly Detection Scheme for Data Stream in Cold Chain Logistics". In: *PLOS ONE* 20.3, e0315322. DOI: 10.1371/journal.pone.0315322. URL: https://doi.org/10.1371/journal.pone.0315322.

APPENDIX A

Technology Stack

Table A.1: Project Technology Stack and Dependency Specifications.

Component Layer	Libraries / Frameworks	Functionality in Project
Core Programming	Python, NumPy, Pandas	Core numerical arrays, tabular transformations, and chronological data handling.
Machine Learning	Scikit-Learn ($\geq$1.3.0), joblib	Isolation Forest anomaly detector, model persistence.
Deep Learning	TensorFlow ($\geq$2.15.0)	LSTM-Autoencoder architecture, training on Google Colab T4 GPU, CPU inference.
Discrete-Event Simulation	SimPy ($\geq$4.1.0)	Three-stage (Source $\rightarrow$ Transit $\rightarrow$ Destination) digital-twin replay engine.
Visualization	Matplotlib ($\geq$3.8.0), Seaborn ($\geq$0.13.0)	Precision-recall curves, reconstruction-error timelines, and lead-time comparison charts.
Interactive Dashboard	Streamlit ($\geq$1.33.0), Altair ($\geq$5.0.0)	Reactive web UI, Guided Tour and Free Explore playback modes.
Data Acquisition	kagglehub, kaggle	Programmatic download of the structural-proxy IoT telemetry dataset.
Notebooks & Testing	Jupyter, nbformat, nbclient, nbconvert, ruff	Milestone notebooks (01–06), code-quality linting, and the D33 buffer pre-warming regression test.

APPENDIX B

Repository and Reproduction Instructions

The complete source code, evaluation pipeline, Architectural Decision log, and dashboard application are version-controlled and publicly accessible:

GitHub Repository: https://github.com/Krishna200608/coldchain-ews-twin

B.1 Environment Setup and Pipeline Execution

To reproduce the full pipeline from scratch:

1. **Clone Repository and Install Dependencies:**

```
git clone https://github.com/Krishna200608/coldchain-ews-twin.git
cd coldchain-ews-twin
python -m venv .venv
source .venv/bin/activate
# Windows: .\.venv\Scripts\activate
pip install -r requirements.txt
```

2. **Execute Data Acquisition, Injection, and Baseline Pipeline:**

```
python src/data_acquisition.py
python src/preprocessing.py
python src/anomaly_injection.py
python src/baseline_and_labels.py
```

3. **Train and Evaluate Isolation Forest:**

```
python src/isolation_forest_model.py
python src/if_evaluation.py
```

4. **Prepare LSTM Windows, Train on Colab T4 (see `colab/lstm_train.ipynb`), then Evaluate:**

```
python src/lstm_prep.py
# Train on Colab (T4 GPU), then pull models/lstm_{out,in}.h5
python src/lstm_evaluation.py
```

5. **Run the Digital-Twin Replay and Cross-Check:**

```
python src/twin_monitors.py
python src/twin_crosscheck.py
```

6. **Run the Test Suite:**

```
pytest tests/ -v
```

7. **Launch the Interactive Local Dashboard:**

```
streamlit run dashboard/app.py
```